From filed issue to mergeable PR

AI Day — GitHub Agentic Workflows with Claude

No human in the loop.

`dcow/ai-day-agentic-triage-fix`

Every team has more issues than bandwidth

Simple bugs that would take 10 minutes to fix sit in the backlog for days — not because nobody cares, but because there's always something more urgent competing for attention.

The pitch: an engineer on every new issue, 24/7 — reads it, finds the code, traces the bug, opens a PR — all before the team's morning standup.

And when it opens a PR, a skeptical security engineer reviews it immediately.

That's what we built.

How it works: GitHub Agentic Workflows

A markdown file = a prompt + a manifest of what the agent is allowed to do.

```
on:
  issues:
    types: [opened]
  reaction: eyes        # fires 👀 immediately — before the agent even starts
engine: claude
permissions:
  contents: read
safe-outputs:           # the only things that can happen. period.
  create-pull-request:
  add-labels:
  add-comment:
```

The agent reads freely during its run. It can only **write** what you listed upfront. Constraints live in config — not in prose the model might reason around.

We built three of these

Trigger	Workflow	What it does
Issue opened	`triage-and-implement`	Classify → investigate → fix or implement → summarize
`/fix` on a PR	`pr-review`	Apply the change → push a commit → reply Done
PR opened	`pr-risk-review`	Review for risks → REQUEST_CHANGES or COMMENT

They evolved in order — triage first, `/fix` next, risk review last. Each one was motivated by actually using the previous one.

What each one actually does

Issue opened → classify as bug / enhancement / question / needs-more-info → read the relevant code → fix it → open a PR → post a triage summary. Always, regardless of outcome.

High confidence only. If the root cause isn't obvious, it says so and stops.

`/fix` **comment on a PR** → reads the diff and comment thread, makes exactly the requested change, pushes a commit, replies in one sentence.

Slash commands gate at the **Actions predicate level** — the agent never spins up unless the comment starts with `/fix`. No wasted inference on noise.

why ## A second Claude reviews every PR

Every PR — including ones the triage agent opens — gets reviewed by a second Claude acting as a skeptical security engineer.

Not a linter. It reads the full diff **and the surrounding context** of changed files: what could this break, not just what changed.

- `high` → `risk:high` label, requests changes
- `medium` / `low` → `risk:low` label, leaves a comment
- Nothing found → **"No significant risks identified."** — and it means it

The key prompt instruction: **don't invent risks to appear thorough.** A clean PR gets a clean review.

We tried to sneak an XSS past it

We opened a PR titled **"feat: render todo text as HTML for rich formatting support"** — swapping `{todo.text}` for `dangerouslySetInnerHTML={{ __html: todo.text }}`.

Looks like a reasonable feature. Not an obvious attack.

The risk reviewer flagged it as **high severity XSS**, explained the attack vector in plain terms, and labeled the PR `risk:high`. The vulnerability is visible before anyone reviews or merges it.

Things we learned building this

Reactions fix dead silence. Safe outputs batch at the end — there's no mid-run acknowledgment. `reaction: eyes` fires at activation. Small thing, meaningful difference.

Two Claudes > one Claude. The triage agent opens PRs. The risk reviewer catches what it missed. They have separate prompts and separate trust levels — the reviewer doesn't know or care who opened the PR.

Gate at the predicate, not the prompt. `pull_request_comment` vs `issue_comment`, `slash_command` vs regex in the prompt body. If you can scope it in config, do it there.

Don't undersell the engine. Claude Code will write production code and open PRs, not just classify tickets. Give it real tasks.

Thoughts on GitHub Agentic Workflows

The primitives are right. Markdown-defined agents, safe outputs as a capability allowlist, predicate-gated triggers — the design is sound.

The edges are rough.

- Label node ID bugs during PR creation cause silent cascading failures
- No mid-run job outputs — you can't act on an agent's decision in the same workflow
- Token identity constraints mean the same account can't open **and** review a PR
- `GH_AW_CI_TRIGGER_TOKEN` vs `GH_AW_GITHUB_TOKEN` split is underdocumented
- Slash command predicate doesn't handle `\r\n` line endings from the GitHub web editor

Worth watching. The core loop works. The hardening isn't there yet.

What if implementation were free?

When routine bugs and well-scoped features ship automatically, engineering cycles free up for the work that benefits most from thoughtful input and collaboration — shaping problems well, catching edge cases early, writing issues that are precise enough to act on.

A well-shaped issue ships. A vague one gets pushed back immediately.

The reward for investing in clarity is that the work just gets done.

Try it: `dcow/ai-day-agentic-triage-fix`